

Programming assignments#2

Red-Black Tree

1. Description

You are tasked with finding the number of subarrays (continuous segments of the array) whose sums fall within a specified inclusive range [**lower**, **upper**].

Each subarray is defined by a pair of indices (i, j) where $0 \leq i \leq j < n$ (where n is the length of given array), and the sum is calculated as:

$$\text{Sum} = \text{array}[i] + \text{array}[i + 1] + \dots + \text{array}[j]$$

Please implement an efficient algorithm using a Red-Black Tree to determine the number of subarrays (i, j) such that:

$$\text{lower} \leq \sum_{k=i}^j \text{array}[k] \leq \text{upper}$$

Example 1:

Input: **array** = [2, -1, 1], **lower** = -1, **upper** = 1

Output: 4

Explanation:

There are four subarrays whose sums fall within the inclusive range [-1, 1].

1. Subarray (0, 1)

Elements: [2, -1], Sum: 1

2. Subarray (1, 1)

Elements: [-1], Sum: -1

3. Subarray (1, 2)

Elements [-1, 1], Sum: 0

4. Subarray (2, 2)

Elements: [1], Sum: 1

Example 2:

Input: **array** = [0], **lower** = 0, **upper** = 0

Output: **1**

Explanation:

There is one subarray whose sum fall within the inclusive range [0, 0].

1. Subarray (0, 0)

Elements: [0], Sum: 0

2. Input/Output Format

The input will consist of two lines:

- First Line: A sequence of integers representing the array. The integers are space-separated.
- Second Line: Two integers representing lower and upper, separated by a space.

```
2 -1 1
-1 1
```

The output should consist of an integer, representing the number of subarrays whose sums fall within the inclusive range [**lower**, **upper**].

```
4
```

3. Input Constraints

- $1 \leq \text{length of array} \leq 10^5$
- $-2^{31} \leq \text{element of array} \leq 2^{31} - 1$
- $-10^5 \leq \text{lower} \leq \text{upper} \leq 10^5$

4. Program

- Program in C or C++.
- You are required to **implement the Red-Black Tree yourself**. Using built-in or library-provided tree data structures (e.g., `std::set`, `std::map` in C++) is **not allowed**.

5. Grading

1. TA will randomly check your code. If it is found that you did not use a Red-Black Tree, you will receive 0 points. If you did not implement the Red-Black Tree yourself (i.e., if you used built-in or library-provided tree data structures), 40 points will be deducted from your grade.
2. Score:
 - Q1~Q50: Each contributes 2%
 - Q1~Q5 are open case
 - Q6~Q50 are hidden case
3. The score of questions is evaluated by the OJ system. TA will evaluate your grades based on the most recent version of your submissions.

6. E3 Submission

1. Submit your source file to the E3 system [Student_ID_Number]_hw2.cpp (.c)
2. Please submit the source code of your latest submission on the OJ. Please make sure that all characters of the filename are in lowercase.
3. Remember the submission rules mentioned above, or you will be penalized by -15 points on your grade.

7. Policy

1. The upload deadline would be at **23:59 on October 30, 2024**. The score will be **30% off** if your submission is late.
2. Plagiarism: **-100 points**.

8. Problems

1. If you have any problem with this project, please post it on the E3 forum.
2. If you have any private problem, you can send emails to:

chunchih.ee13@nycu.edu.tw